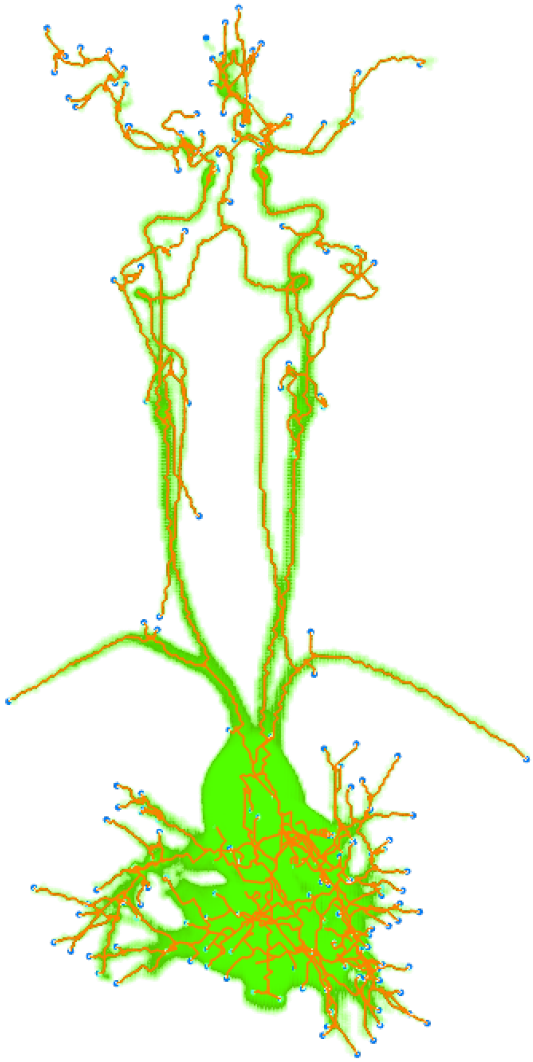
Using Angicart++

Analyzing 3D Radiographic Images of Vasculature to Determine Vessel Properties



Contributors

Dr. Van Savage
Jocelyn J. Shen
David Hunt
Alex Brummer
Anderson Ju

UCLA Department of Biomathematics

Updated 2017

Contents

1 Introduction

Angicart++ is written in C++ and utilizes wxWidgets and OpenGL to create a cross-platform interactive visualization window

Angicart++ works using the following algorithm:

- Reads in the set of PNG images over a range of the directory specified by the user.
- A BinaryVolume "B" is constructed (where the values of the bitset are true in locations that correspond to vasculature)
- Points of "B" are initialized, visualization begins, and a map of points, "pm," is created for the RGBA color of each point to be accessed quickly
- "B" is reduced to its N largest connected components, where N is an integer. The remaining, smaller components are removed.
- N can be chosen so there is some component with no backbone
- Visualization is updated to the set of points comprising the components
- Segmentation:
  - Define a set of spheres to provide a coarse map of the vascular network
    - Spheres defined in cubic voxels, but physical dimensions could be used if voxel asymmetries become common
  - Rough structure of segments is constructed by serially creating disjoint spheres that grow to the size of the vascular segment they occupy
  - Algorithm for sphere quality:
    1. Choose a random point in "B" that is not a seed for a potential sphere
    2. Grow the sphere until the fraction of voxels expanding falls below a threshold (ie. 0.16)
    3. If the sphere doesn't find sufficient voxels in the expanding region, a disjoint sphere labeled valid is returned
    4. If the expanding region of the sphere includes a point already belonging to another sphere, then the current seed is labeled invalid and abandoned as a viable seed
    5. If necessary, the centers of the valid spheres will reposition, and the center of mass of the vascular section will be taken as the new seed, and the old seed will be abandoned.
    6. Once the seed stops moving, or the sphere's radius doesn't increase, then the seed is accepted.
    7. Search for valid spheres until a specified number of seeds fail consecutively
    8. With more consecutive failures, the model will re-segment
  - Once spheres are determined, the vascular segments are connected into single components between spheres, at the same time mapping which spheres are adjacent to which segment.

- Removal Algorithm:
  1. The model is now segmented into two spaces: intra-sphere spaces (within the particular sphere's radius of each sphere seed) and inter-sphere spaces (singly connected components joining spheres)
  2. False tips are caused by spheres not filling a vessel
  3. Spheres: False times identified by finding spheres with only a single connection and a singly connected component between intra-sphere and inter-sphere spaces.
  4. Connections: Any inter-sphere space including fewer than 2 spheres is removed
- Define critical voxels as the center of mass of each singly-connected component between intra and inter sphere spaces
- Each critical voxel is associated with its connection, which is in turn paired with appropriate spheres when finding raw backbones
- Critical voxels define the "tips" for the skeletonization of each sphere and connection
- Skeletonization uses erosion to determine centerlines farthest from the outer edge of the vessel
- Raw backbones are sequentialized and stiched back together based on critical vertebrae so that they have ¿ 3 segments
- Process repeated until each distinct segment has its own "backbone" (a vector of position indexes, stored in vector¡vector¡unsigned int¿¿)
- Empty backbones are removed
- Tips re-visited to ensure their backbones extend to their full lengths
- Extend tips by:
- Identifying the vessel voxels nearest to the current backbone
  * Identify neighborhood of backbones near target backbone
  * Identifying target segments follows the rule that they must be able to extend the neighboring adjacent segments until the length of all paths in the adjacent segments is at least twice as long as the target backbone
- Now that the neighborhood of the backbones is adequately defined, frontiers from each backbone will expand until the target segments are fully defined
- Once target segments are known, the voxel of the target backbone is used in the erosion algorithm with the tip of the original target backbone
- Stitching extensions onto all the tips, the backbones fill up the vascular structure
- Locally, tips are refined
- Backbones are segregated into their connected components and stores in global variables for the visualization

2 Installation

2.1 GitHub

https://github.com/andersonju9797/AngicartCPlusPlus

2.2 Mac OSX

Mac OSX hints:

- Update OS (Angicart++ was tested on 10.11.4 El Capitan)
- Install/update Xcode (Angicart++ was tested on Xcode v 7.3)
- Install Xcode command line using: `code-select --install`
- Install Homebrew using: `brew install wxmac`

2.3 Windows

Windows hints:

- Update OS (Angicart++ was tested on Windows 7)
- Update Visual Studio (Angicart++ was tested on VS Express 2013)
- Install wxWidgets (Check https://www.wxwidgets.org/docs/ for help)
- F5 to build and run
- To run without wxWidgets interface, change `#define TRY_WX 1` to `#define TRY_WX 0` on line 6 of `wxMinSurfTests.h`

For specifying another data set to analyze, nine pieces of information (given below in order) are needed:

1. string image_directory
2. int image_start_inclusive
3. int image_end_inclusive
4. string output_filename_base
5. double voxel_dimension_x
6. double voxel_dimension_y
7. double voxel_dimension_z
8. string length_unit_name
9. double normalized_intensity_threshold

3 Instructions

Using the interactive visualization:

Command	Action
Drag cursor	Change position of camera
Scroll mouse wheel	Change position of camera
Hold Shift + Drag cursor	Change position of focus point
Hold Shift + Scroll mouse wheel	Change position of focus point
Click	Defines line from camera through click location
Double-click	Defines point of nearest approach
L	Labels hidden or toggled in size

Output summary of segment properties:

1. First format includes properties of a segment and its adjacent segments
2. Second output chooses root segment for each connected component and reports structure as tree
    - Might yield only 1 child due to a loop

4 Future Directions

- Speed and efficiency
- Interface interaction taking into account user error
- Problems when holes or segment lengths are comparable to diameter
- Highlight suspicious segments

5 Acknowledgements

I would like to thank David Hunt for providing program notes to Angicart++, which helped me understand the theory behind the software, and is the premise of this manual.